

SysHealth — live demo runbook

Memory pressure across four EC2 instance sizes. Everything below runs from `~/syshealth/terraform`.

1 Fifteen minutes before

Do all of this before anyone is watching. The fleet needs three minutes to boot and calibrate, and you want one rehearsal run in hand.

Bring the fleet up

```
cd ~/syshealth && git pull
./terraform/relaunch.sh
```

Checks your credentials, that the AMI is pinned and the demo tag exists, rewrites `ssh_cidr` to whatever address you are on now, then applies and waits until all four sizes report.

Expect: a table of four sizes, all `online=yes`, then a dashboard URL.

Recalibrate on quiet boxes

Baselines are measured for 60 seconds at first boot, right after package installation — so they can be recorded too high. Every state on the dashboard is relative to them, so redo it while nothing is loaded.

```
cd terraform
./recalibrate.sh

# if your key is somewhere else:
KEY=~/.ssh/your-key.pem ./recalibrate.sh
```

Runs in parallel, about 80 seconds, then prints the new baselines. They should all be the same order of magnitude — one far above the rest means that box was still busy, so wait a minute and run it again. Do not stress anything while it runs, or the load becomes the baseline.

Rehearse once, then let it settle

```
curl -X POST "$(terraform output -raw dashboard_url)/run-stress"
sleep 45 && ./check.sh
```

Expect: micro and small CRITICAL, medium and large HEALTHY. If not, see section 4 before your teacher arrives.

Then wait about two minutes for pressure to fall back to zero, so the live run starts from a flat line.

2 The demo

Step 1 — Open the dashboard

```
terraform output -raw dashboard_url    # paste into the browser
```

Four cards, all HEALTHY, all green dots, a flat line across the chart.

Say: Four EC2 instances, identical software, different sizes: micro at 1 gigabyte up to large at 8. Each one reads the kernel's own pressure counter every five seconds and reports here. Right now they are all idle.

Step 2 — Explain what is being measured

Say: This is PSI — pressure stall information, straight from the Linux kernel. It is the percentage of time processes were stalled waiting for memory. Not how much memory is used, but how much the machine is struggling to get it. A box can be at 90 percent memory and perfectly happy; this measures the pain, not the level.

Step 3 — Load all four at once

Press **Stress all** on the dashboard. Or from the terminal:

```
curl -X POST "$(terraform output -raw dashboard_url)/run-stress"
```

All four are asked in parallel, within a few milliseconds, so the comparison is fair. Each runs the same job: two workers, 800 MB each, for 60 seconds.

Say: Same workload, same moment, four different machines. Watch which ones feel it.

Step 4 — Watch the separation (60 to 90 seconds)

Instance	RAM	What happens
t3.micro	1 GB	Spikes hard, then the OOM killer ends the job
t3.small	2 GB	Climbs and stays there — the worst sustained case
t3.medium	4 GB	Barely moves
t3.large	8 GB	Flat

Say: The 1.6 gigabyte job fits comfortably on medium and large — they never notice it. On small it fits, but only just, so the kernel spends the whole minute reclaiming memory. That is what pressure looks like.

Step 5 — Show one machine on its own

Click **t3.small** in the Instance toggle. Two dashed threshold lines appear.

Say: These thresholds are per machine, calculated from that box's own idle baseline — two times for degraded, five times for critical. A micro idles differently from a large, so a single fixed number would be wrong for both. That is why the lines only show when one machine is selected.

Step 6 — The result

Say: The finding is that t3.medium, at 4 gigabytes, is the smallest instance that stays healthy under this workload. Anything smaller degrades. That is a sizing decision made from measurement instead of guesswork.

3 Questions you will get

“Why is small worse than micro? Shouldn't the smallest be worst?”

This is the best question anyone can ask you, and you have the evidence.

Say: It does not degrade — it fails. The job needs 1.6 gigabytes; micro has 1 plus half a gigabyte of swap. It does not fit, so the kernel kills the job outright. Pressure collapses because the work stopped, not because the machine coped. Micro's score understates the problem. Small is the worst case you can actually observe: big enough to accept the work, too small to finish it.

Proof, live, if they want it:

```
MICRO=$(terraform output -json instances | python3 -c \
    "import json,sys; print(json.load(sys.stdin)['t3.micro']['public_ip'])")

ssh -i $KEY ubuntu@$MICRO \
    'sudo journalctl -k --since "20 min ago" | grep -i "killed process"'
```

Prints the kernel's own record of killing the stress job, with a timestamp that matches the spike on the chart.

“Why not just watch memory usage?”

Say: Usage tells you a number, not whether it hurts. A box sitting at 95 percent with nothing to reclaim is fine. A box at 60 percent thrashing is not. PSI measures time lost to waiting, which is the thing users actually feel.

“How do the health states work?”

Say: Each agent measures its own idle baseline for 60 seconds when it starts. After that, two times baseline is degraded, five times is critical — and it has to hold for three consecutive samples, fifteen seconds, before the state changes. That stops one momentary spike from being reported as a failure.

“What is it built on?”

Say: Python and Flask. The agent reads /proc/pressure/memory and /proc/vmstat. The chart is hand-written SVG, so there is no charting library and nothing loaded from the internet. The whole fleet is Terraform — one command builds all five machines.

“Could this run in production?”

Say: The measurement side, yes. Two things would need work first: history is kept in memory and is lost if the server restarts, so it would need a real datastore. And the dashboard has no authentication — right now it is locked to a single IP address, which is fine for a demo and not for anything real.

4 If something breaks

The dashboard will not load

Almost always your IP address changed — a different network, or the ISP moved you. The security group still allows the old one. It fails silently and looks exactly like a server that never started.

```
./terraform/relaunch.sh    # detects it, fixes only the firewall rule
```

Expect: 0 to add, 3 to change, 0 to destroy. No instance may appear in that plan.

A size is missing from the dashboard

```
./check.sh                # names which sizes are not reporting
terraform output instances # public IP of each
ssh -i $KEY ubuntu@<ip> 'sudo tail -30 /var/log/syshealth-bootstrap.log'
ssh -i $KEY ubuntu@<ip> 'sudo journalctl -u syshealth -n 30'
```

An agent that is running but cannot reach the dashboard says so in its own log: push to http://... failed.

Everything says HEALTHY even under load

A baseline was recorded while the box was busy, so the floor is too high. States are ratios against that floor, so the machine reads calm while it struggles.

```
./recalibrate.sh          # prints every baseline when it finishes
```

One baseline far above the others is the culprit. Let the fleet sit quiet for a minute first, or you will record the load again.

Nothing works and you must present anyway

There is a local simulation that needs no AWS and no network. It serves the real dashboard over a simulated four-size fleet, and every page is marked DEMO.

```
cd ~/syshealth
sudo apt install -y python3-flask python3-requests # first time only
npm run dev          # then open http://127.0.0.1:5000
```

Say plainly that it is simulated. The states and thresholds still come from the same analysis code that runs on the real instances — only the sensor is faked.

5 Afterwards

Save the evidence before you destroy anything. History lives in the server's memory and goes with it.

```
curl -s "$(terraform output -raw dashboard_url)/series?limit=720" \
  > ~/syshealth/demo-evidence-$(date +%F).json

terraform destroy

aws ec2 describe-instances --region ap-south-1 \
  --filters Name=instance-state-name,Values=running,pending \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType]' --output text
```

Expect: empty output from the last command. That means billing has stopped.

Five instances cost roughly US\$5 a day and bill until destroyed.